

Cálculo Numérico MS211K - 2S/2020

Atividade 1

Eduardo Kleber Silva 138276
Guilherme Fratin Medina 135954
Luís Roberto Marques Junior 173216

O objetivo da atividade 1 é trabalhar com sistema de ponto flutuante (SPF) e aproximações, avaliando quão boas elas são baseado no valor dos erros envolvidos, sejam eles absolutos ou relativos. Para isso foi utilizada a fórmula indicada, a série descrita por

$$S = \sum_{k=1}^{\infty} \frac{1}{(2k-1)^2} \text{ com resultado } S = \frac{\pi^2}{8}$$

Como sabemos, não é possível iterar infinitamente no computador, portanto precisamos adaptar para algo como

$$S|N| = \sum_{k=1}^N \frac{1}{(2k-1)^2}$$

com **N** sendo um número finito grande de iterações tal que $S \approx S|N|$.

Todos os cálculos serão efetuados com precisão simples, portanto esperamos um erro para a aproximação na casa de 1×10^{-7} para termos uma acurácia de resultados aceitável.

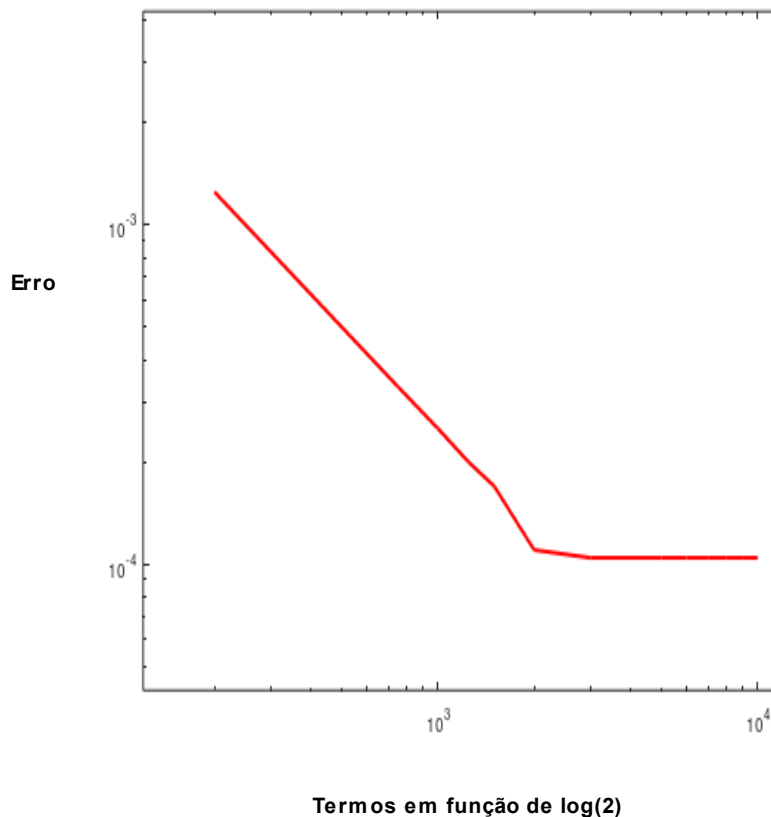
Inicialmente montamos um algoritmo genérico que faz uma soma crescente, podendo ser implementado em qualquer linguagem, que funciona da seguinte forma:

- **soma** $\leftarrow 0$
- **contador** $\leftarrow 1$
- enquanto **contador** $\leq$ **total**
 - soma** $\leftarrow$ **soma** + $\frac{1}{(2k-1)^2}$
 - contador** $\leftarrow$ **contador** + 1

Inicializamos três variáveis **soma**, **contador** e **total**; a primeira tem como função receber a soma de cada uma das iterações, começando com valor inicial 0. A segunda faz a iteração do algoritmo, “contando” as iterações necessárias e a terceira delimita quantas vezes essa iteração acontece.

Fazendo dessa forma, obtemos um erro como valor na grandeza de 1×10^{-4} , que não se altera mais, não importando quantas iterações mais façamos.

Soma Crescente em Log(2)



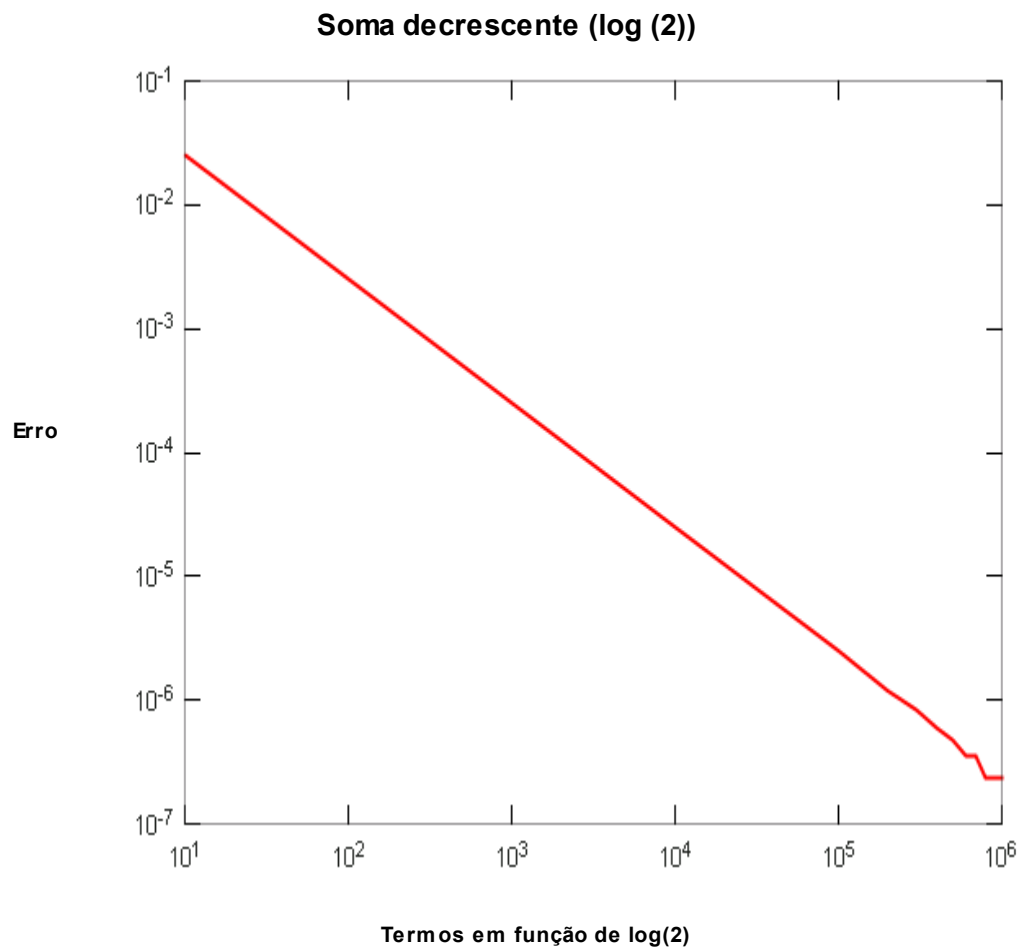
Considerando que a grandeza está bem acima da esperada para ser um erro aceitável, precisamos pensar numa forma de retrabalhar o algoritmo ou adaptá-lo; seguindo a dica que vimos em aula, tentamos inverter o processo, fazendo as somas do maior termo para o menor.

O algoritmo dessa inversão, ou no caso, da soma decrescente:

- **soma** $\leftarrow 0$
- **contador** $\leftarrow$ **total**
- enquanto **contador** ≥ 1
 - soma** $\leftarrow$ **soma** + $\frac{1}{(2k-1)^2}$
 - contador** $\leftarrow$ **contador** - 1

Reutilizando as variáveis da primeira situação só tivemos que ajustar algumas instâncias; no caso, para a soma acontecer de forma decrescente, a variável **contador** precisa ser inicializada com o valor máximo de iterações e ir diminuindo até a “última” soma, que no caso, é o valor mais baixo, 1.

Dessa forma passamos a obter um erro de uma grandeza muito menor, dentro da esperada para o erro, na casa de 1×10^{-7} .



Com todo esse processo conseguimos fazer uma aproximação muito boa a partir do segundo algoritmo; o primeiro algoritmo recai num problema no qual somamos valores de ordens muito distintas. Depois de quase duas mil somas, o valor guardado na variável **soma** era muito superior ao valor super pequeno de $f(k) = \frac{1}{(2k-1)^2}$ a partir de $k \approx 2000$ e termos em diante. Já no segundo algoritmo, garantimos que a soma dos termos, por menor que fossem, sempre fosse feita entre parcelas de mesma grandeza ou muito próxima, não acontecendo nenhuma perda de dígitos significativos em nenhuma das instâncias, resguardando o resultado final.

Os códigos de implementação do Octave juntamente com os gráficos gerados estão anexados na entrega da tarefa, juntamente com as cláusulas antifraude de cada integrante do grupo.